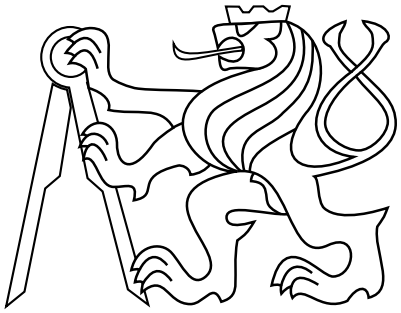
Czech Technical University in Prague

Faculty of Electrical Engineering
Department of Cybernetics
Multi-robot Systems



My Thesis Title
Can Span Multiple Lines

Bachelor's Thesis

John Smith

Prague, January 2021

Study programme: Electrical Engineering and Information Technology
Branch of study: Artificial Intelligence and Biocybernetics

Supervisor: Ing. My Supervisor, Ph.D.

Acknowledgments

Firstly, I would like to express my gratitude to my supervisor.

Swap this for the Thesis Assignment, when you have it from the department.

Declaration

I declare that presented work was developed independently, and that I have listed all sources of information used within, in accordance with the Methodical instructions for observing ethical principles in preparation of university theses.

Date ....

Abstract

The study of autonomous Unmanned Aerial Vehicles (UAVs) has become a prominent sub-field of mobile robotics.

Keywords Unmanned Aerial Vehicles, Automatic Control

Abstrakt

Výzkum na poli autonomních bezpilotních prostředků (UAV) se stal významným oborem mobilní robotiky.

Klíčová slova Bezpilotní Prostředky, Automatické Řízení

Abbreviations

UAV Unmanned Aerial Vehicle

Contents

1 Introduction **1**
1.1 Related works . . . 1
1.2 Contributions . . . 1
1.3 Mathematical notation . . . 1

2 Methodology **2**
2.1 Binary Mask Processing . . . 2
2.1.1 Binary Mask Creation . . . 2
2.1.2 Euclidean Distance Transform . . . 2
2.1.3 Signed Distance Field . . . 3
2.1.4 Binary Mask Smoothing . . . 4
2.2 Topology Extraction from Binary Masks . . . 5
2.2.1 Binary Mask Skeletonization . . . 5
2.2.2 Contour Extraction of Area Features . . . 7
2.3 Polyline Creation from Topological Pixels . . . 7
2.3.1 Pixel Classification . . . 7
2.3.2 Polyline Tracing Algorithm . . . 7
2.3.3 Edge Cases . . . 9
2.4 Polyline Post-processing . . . 9
2.4.1 Polyline Stitching Algorithm . . . 9
2.4.2 Polyline Simplification Using Douglas–Peucker Algorithm . . . 11
2.5 Spline Construction from Polylines . . . 12
2.5.1 Hermite and Bézier Curve Representations . . . 12
2.5.2 Catmull–Rom Spline Construction . . . 13
2.5.3 Curve Subdivision Using de Casteljau's Algorithm . . . 15

3 Conclusion **17**

4 References **18**

A Appendix A **19**

1 Introduction

First, introduce the reader to the research topic. Start with the most general view and slowly converge to the particular field, sub-field, and the challenges you face. You can cite others' work here **baca2021mrs**.

1.1 Related works

This section should contain related state-of-the-art works and their relation to the author's work. We usually cite the original works like this **benallegue2008high**. You can also cite multiple papers at once like this **baca2016embedded**, **baca2021mrs**.

1.2 Contributions

This section should describe the author's contributions to the field of research.

1.3 Mathematical notation

It is a good practice to define basic mathematical notation in the introduction. See Table 1.1 for an example.

$\mathbf{x}$, $\boldsymbol{\alpha}$	vector, pseudo-vector, or tuple
$\hat{\mathbf{x}}$, $\hat{\boldsymbol{\omega}}$	unit vector or unit pseudo-vector
$\hat{\mathbf{e}}_1, \hat{\mathbf{e}}_2, \hat{\mathbf{e}}_3$	elements of the *standard basis*
$\mathbf{X}, \boldsymbol{\Omega}$	matrix
$\mathbf{I}$	identity matrix
$x = \mathbf{a}^\intercal \mathbf{b}$	inner product of $\mathbf{a}$, $\mathbf{b} \in \mathbb{R}^3$
$\mathbf{x} = \mathbf{a} \times \mathbf{b}$	cross product of $\mathbf{a}$, $\mathbf{b} \in \mathbb{R}^3$
$\mathbf{x} = \mathbf{a} \circ \mathbf{b}$	element-wise product of $\mathbf{a}$, $\mathbf{b} \in \mathbb{R}^3$
$\mathbf{x}_{(n)} = \mathbf{x}^\intercal \hat{\mathbf{e}}_n$	n^{th} vector element (row), $\mathbf{x}, \mathbf{e} \in \mathbb{R}^3$
$\mathbf{X}_{(a,b)}$	matrix element, (row, column)
x_d	x_d is *desired*, a reference
$\dot{x}, \ddot{x}, \dddot{x}, \ddddot{x}$	1^{st}, 2^{nd}, 3^{rd}, and 4^{th} time derivative of x
$x_{[n]}$	x at the sample n
$\mathbf{A}, \mathbf{B}, \mathbf{x}$	LTI system matrix, input matrix and input vector
SO(3)	3D special orthogonal group of rotations
SE(3)	*SO(3)* $\times \mathbb{R}^3$, special Euclidean group

Table 1.1: Mathematical notation, nomenclature and notable symbols.

2 Methodology

2.1 Binary Mask Processing

Binary Mask Creation

A segmentation image is converted into a binary mask by selecting all pixels whose color lies within a specified range. This range can be adjusted by the user to obtain finer control over binary mask creation.

The result is a binary image in which white pixels, referred to as foreground, are assigned the value true, while black pixels, referred to as background, are assigned the value false. We refer to clusters of white or black pixels as regions.

Euclidean Distance Transform

The Euclidean distance transform, referred to as EDT, assigns to each pixel the Euclidean distance to the nearest pixel of a specified set. This can be used for extracting widths from binary masks.

In our implementation, the cost function f is chosen so that distances are computed from foreground pixels to the nearest background pixel. This is determined by which pixels are assigned the values 0 and ∞ in the cost function.

This process can be generalized to transform a cost function on a grid using spatial information [2], such as using distance function other than Euclidean, or n-dimensional grids.

We define cost function as $f(q)$:

$$f(q) = \begin{cases} \infty & \text{if } q \text{ is a foreground pixel,} \\ 0 & \text{otherwise,} \end{cases} \tag{2.1}$$

where q is a pixel of the binary mask. The distance transform of f is denoted by $\mathcal{D}_f$ and defined as

$$\mathcal{D}_f(p) = \min_{q \in \mathcal{G}} \left(d(p,q) + f(q) \right), \tag{2.2}$$

where $\mathcal{G}$ is the grid representing the binary mask, $p \in \mathcal{G}$, and $d(p,q)$ is a distance function.If $d(p,q)$ is chosen as Euclidean distance, then $\mathcal{D}_f$ is called the Euclidean distance transform.

Applying EDT to the binary mask yields an unsigned distance field. The value at each element of the distance field is the distance from the corresponding pixel in the binary mask to the nearest background pixel. This can be seen in definition of function $f(q)$, where its value for black pixels is zero, meaning $\mathcal{G}$ computes distance of a white pixel to a black pixel. Consequently, the value of every background pixel in this field is 0.

In our application, this algorithm is used to extract the widths of roads or rivers by computing EDT of foreground pixels. These widths can then be converted to approximate real-world measurements. The precision of this approach depends on the accuracy and resolution of the binary mask.

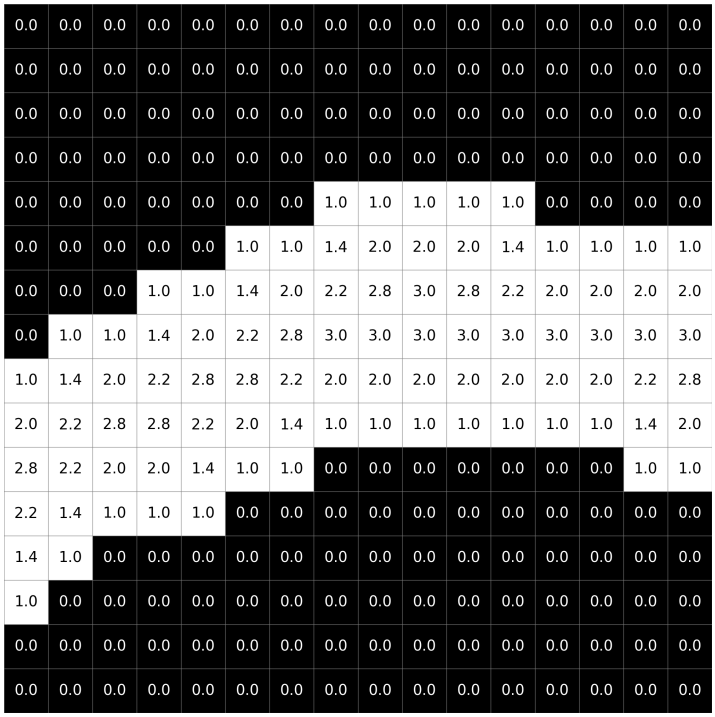


Figure 2.1: Visual unsigned distance field of a binary mask.

Signed Distance Field

Signed distance field, or SDF, is a field in which the value corresponding to each pixel of binary mask represents the distance from that pixel to the nearest boundary between foreground and background region. Unlike the unsigned distance field, the SDF stores meaningful values on both sides of the foreground boundary. To differentiate between the two colors, we denote the distances of background pixels as negative.

First we define distance transform $\mathcal{D}_g$ for background pixels by inverting the cost function, so we compute distances of black pixel to a white pixel. The cost function $g(q)$ is

$$g(q) = \begin{cases} \infty & \text{if } q \text{ is a background pixel,} \\ 0 & \text{otherwise,} \end{cases} \tag{2.3}$$

where q is a pixel of the binary mask. The distance transform of g is denoted by $\mathcal{D}_g$ and defined as

$$\mathcal{D}_g(p) = -\min_{q \in \mathcal{G}} \left(d(p,q) + g(q) \right), \tag{2.4}$$

where $\mathcal{G}$ is the grid representing the binary mask, $p \in \mathcal{G}$, and $d(p,q)$ is the Euclidean distance.

Signed distance field has the same dimensions as the initial binary mask. The SDF value v at pixel p is defined as the sum of these two distance transforms

$$v = \mathcal{D}_f(p) + \mathcal{D}_g(p), \tag{2.5}$$

where p is a pixel on binary mask. When p is a foreground pixel, $\mathcal{D}_f(p)$ is positive, while $\mathcal{D}_g(p) = 0$. Conversely, when p is a background pixel, $\mathcal{D}_f(p) = 0$ and $\mathcal{D}_g(p)$ in a negative value.

-6.7	-6.3	-5.8	-5.4	-5.0	-4.5	-4.1	-4.0	-4.0	-4.0	-4.0	-4.0	-4.1	-4.5	-5.0	-5.0
-5.8	-5.4	-5.0	-4.5	-4.1	-3.6	-3.2	-3.0	-3.0	-3.0	-3.0	-3.0	-3.2	-3.6	-4.0	-4.0
-5.0	-4.5	-4.1	-3.6	-3.2	-2.8	-2.2	-2.0	-2.0	-2.0	-2.0	-2.0	-2.2	-2.8	-3.0	-3.0
-4.1	-3.6	-3.2	-2.8	-2.2	-2.0	-1.4	-1.0	-1.0	-1.0	-1.0	-1.0	-1.4	-2.0	-2.0	-2.0
-3.2	-2.8	-2.2	-2.0	-1.4	-1.0	-1.0	1.0	1.0	1.0	1.0	1.0	-1.0	-1.0	-1.0	-1.0
-2.2	-2.0	-1.4	-1.0	-1.0	1.0	1.0	1.4	2.0	2.0	2.0	1.4	1.0	1.0	1.0	1.0
-1.4	-1.0	-1.0	1.0	1.0	1.4	2.0	2.2	2.8	3.0	2.8	2.2	2.0	2.0	2.0	2.0
-1.0	1.0	1.0	1.4	2.0	2.2	2.8	3.0	3.0	3.0	3.0	3.0	3.0	3.0	3.0	3.0
1.0	1.4	2.0	2.2	2.8	2.8	2.2	2.0	2.0	2.0	2.0	2.0	2.0	2.0	2.2	2.8
2.0	2.2	2.8	2.8	2.2	2.0	1.4	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.4	2.0
2.8	2.2	2.0	2.0	1.4	1.0	1.0	-1.0	-1.0	-1.0	-1.0	-1.0	-1.0	-1.0	1.0	1.0
2.2	1.4	1.0	1.0	1.0	-1.0	-1.0	-1.4	-2.0	-2.0	-2.0	-2.0	-2.0	-1.4	-1.0	-1.0
1.4	1.0	-1.0	-1.0	-1.0	-1.4	-2.0	-2.2	-2.8	-3.0	-3.0	-3.0	-2.8	-2.2	-2.0	-2.0
1.0	-1.0	-1.4	-2.0	-2.0	-2.2	-2.8	-3.2	-3.6	-4.0	-4.0	-4.0	-3.6	-3.2	-3.0	-3.0
-1.0	-1.4	-2.2	-2.8	-3.0	-3.2	-3.6	-4.1	-4.5	-5.0	-5.0	-5.0	-4.5	-4.1	-4.0	-4.0
-2.0	-2.2	-2.8	-3.6	-4.0	-4.1	-4.5	-5.0	-5.4	-5.8	-6.0	-5.8	-5.4	-5.1	-5.0	-5.0

Figure 2.2: Visual Signed distance field of a binary mask.

Binary Mask Smoothing

Binary mask smoothing is achieved by applying Gaussian blur to the computed signed distance field. The blurred field is then classified by a threshold. Values above a this threshold are classified as foreground, while the remaining values are classified as background. When applied to very thin foreground regions, this process may disrupt their continuity.

Gaussian function for two dimensions is defined as

$$G(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right), \tag{2.6}$$

where parameter σ determines the blur strength. Increasing σ results in stronger smoothing, while decreasing produces weaker smoothing.

To blur an image $I(x, y)$ we compute its convolution with Gaussian function

$$I_{\text{blurred}}(x, y) = (I * G)(x, y) = \sum_u \sum_v I(x - u, y - v)G(u, v). \tag{2.7}$$

In discrete implementation we use kernel of size $(2R + 1) \times (2R + 1)$, where R is a radius generally chosen as

$$R = \max\left(1, \lceil 3\sigma \rceil\right), \tag{2.8}$$

because the value of Gaussian function outside the interval $[-3\sigma, +3\sigma]$ is relatively small.

If we use computed SDF of binary mask as our image, then we get resulting blurred SDF as

$$\text{SDF}_{\text{blurred}}(x, y) = \sum_{u=-R}^{R} \sum_{v=-R}^{R} \text{SDF}(x - u, y - v)G(u, v), \tag{2.9}$$

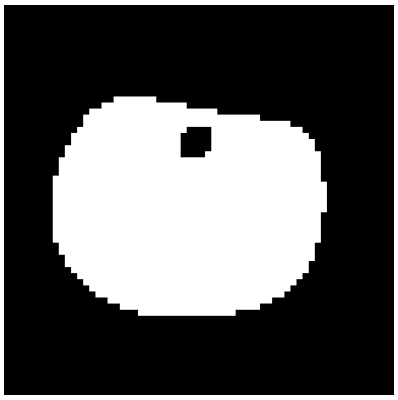
where $\mathrm{SDF}(x, y)$ is an element of SDF at x and y coordinate, and $0 \leq x <$ width and $0 \leq y <$ height.

The user is given option to adjust both Gaussian blur parameter σ the threshold value. Increasing the threshold erodes the resulting mask, whereas decreasing it expands the foreground region.

(a) Binary mask input.

(b) Binary mask smoothing with $\sigma = 5.0$.

(c) Eroded binary mask with threshold $= 1.5$.



(d) Expanded binary mask with threshold $= -1.0$.

Figure 2.3: Examples of smoothed binary masks. Input binary mask (a) is smoothed to (b) with Gaussian parameter $\sigma = 5.0$. Smoothed binary mask (b) is eroded (c) with threshold $= 1.5$ and is expanded (d) with threshold $= -1.0$.

2.2 Topology Extraction from Binary Masks

Binary Mask Skeletonization

Thinning of a binary mask, referred to as skeletonization, is a process of removing boundary pixels of a foreground region while maintaining their basic structure, general shape and connectivity. For algorithms used to create binary mask skeleton used for this application, we need to ensure satisfaction of certain criteria:

- The resulting line should be 1 pixel wide with no redundant pixels
- Connectivity of individual components must be preserved

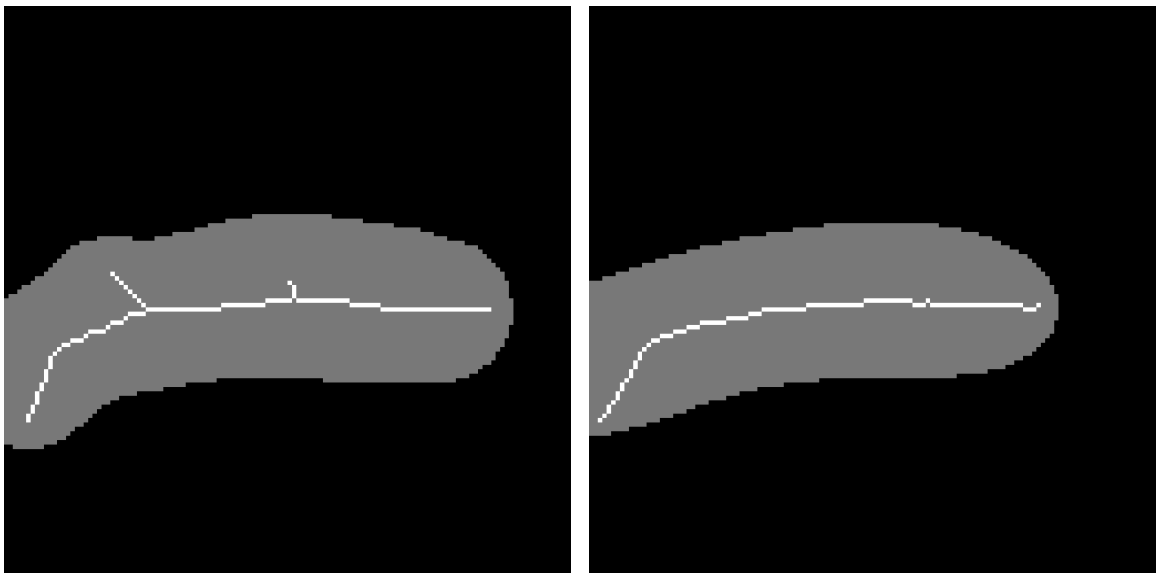
- The skeleton should be approximately centered with respect to the local thickness of the component.
- There should not be substantial gap between skeleton and edge of binary mask, if the foreground region is touching the side.

Many thinning algorithms have been proposed for various applications, one of the popular ones are Guo-Hall algorithm [3] and Zhang-Suen algorithm [7]. Their main advantage is their ability to be parallelized, but both of them do not guarantee desired 1-pixel width without some additional post-processing. A different algorithm was selected [1], which is sequential and guarantees skeleton with no redundant pixels and 1-pixel width.

This algorithm works by performing passes in all four directions and sequentially removes redundant pixels until only the skeleton remains. This results in the skeleton being offset by 1 pixel from center, but this error is insignificant and does not negatively affect the final result.

Disadvantage of skeletonization algorithms is that they are sensitive to binary mask "corners". As the algorithm creates skeleton to be in the middle, when some section of the foreground region is sharper, it can create branch towards this corner. This can be resolved by smoothing the binary mask before performing skeletonization. Despite the mask slightly changing shape due to the smoothing, the general shape of the skeleton is preserved.

As seen on 2.4b, even performing skeletonization on smoothed binary mask can lead to undesirable curving on the skeleton. This level of control can not be achieved by simple binary mask smoothing but other more complex algorithms would be needed. However, this results is acceptable for out application of creating splines, as the user has the option to manually adjust them, if the error is too noticeable.



(a) Skeletonization performed on binary mask

(b) Skeletonization performed on smoothed binary mask

Figure 2.4: Example of resulting binary mask skeleton without (a) smoothing and with smoothing (b). The gray area is the binary mask, either initial or smoothed. Smoothing was performed with $\sigma = 15.0$ and threshold -1.2.

Contour Extraction of Area Features

Extracting contours of binary mask foreground region is used for creating features that fill area, such as biomes or lakes.

Used algorithm cycles through every pixel of binary mask. If the pixel is black, that means in background, we skip it. If the pixel is white, in foreground, we count how many 4-neighbors are black and if there are any, we mark this pixel as contour. Pixels that are 4-neighbors are the pixels directly above, below, left or right of the initial pixel.

This algorithm skips foreground pixels along the edges of binary mask, as there are no background pixels. To create edge contour we count pixels that are outside the binary mask bounds as background.

This algorithm, although simple, reliably detects contours. There are a few scenarios where strictly 1-pixel width of the contours is not ensured. These scenarios are hard to resolve by algorithm as their outcome can rapidly change the shape or size of the contours. We decided to leave the resolution of these situations to user by giving them tools for manual edit.

2.3 Polyline Creation from Topological Pixels

A polyline is represented os an ordered list of points $\mathbf{P}_0, \mathbf{P}_1, ..., \mathbf{P}_n$ where each consecutive pair of points $(\mathbf{P}_i, \mathbf{P}_{i+1})$, where $i = 0, 1, ..., n$, defines one line segment. In this way, the polyline forms a connected piecewise curve composed of the segments

$$(\mathbf{P}_0, \mathbf{P}_1), (\mathbf{P}_1, \mathbf{P}_2), ..., (\mathbf{P}_{n-1}, \mathbf{P}_n).$$

Since the points are ordered, the connectivity of the polyline is given implicitly by their order in the list.

Special case of polyline is a closed polyline. A polyline is considered closed if its first and last point are identical,

$$\mathbf{P}_0 = \mathbf{P}_n. \tag{2.10}$$

Pixel Classification

To create polylines we need to classify topological pixels, determining whether a pixel is an endpoint or a junction. We use a simple classification method based on counting the number of foreground pixels in the 8-neighborhood of the examined pixel.

If the number of foreground neighbors is one, the pixel is classified as an endpoint, since it is connected to only one neighboring pixel. If the number is two, the pixel is considered a regular polyline pixel connecting two neighboring pixels.

A junction pixel has three or more foreground neighbors, although in typical cases it has exactly three. If four or more foreground neighbors are present, this may indicate a more complex local structure, such as a plateau or another shape that can complicate polyline extraction.

Polyline Tracing Algorithm

The extracted topology, whether obtained from a binary mask skeleton or from a contour, is still represented as an unordered set of foreground pixels in the image grid. For

subsequent processing, such as polyline simplification or spline construction, this discrete representation must be converted into one or more ordered polylines.

The goal of the tracing algorithm is therefore to transform the foreground topology pixels into a set of ordered polylines that collectively cover the entire topology. To achieve this, the algorithm uses the previously detected endpoint and junction pixels together with a visited map. The visited map records whether a foreground pixel has already been assigned to a traced polyline, which prevents repeated processing of the same branch. Junction pixels require special handling, since a single junction may belong to multiple output polylines.

The tracing is divided into three steps. First, all branches starting at endpoints are traced until another endpoint or a junction is reached. Second, the remaining branches connecting pairs of junctions are traced. Finally, any still unvisited foreground pixels are assumed to belong to closed contours and are processed separately. This ordering is important, because endpoint-based branches are the least ambiguous, while closed contours can only be identified reliably after all open branches have already been removed.

When tracing from a current pixel, the algorithm examines its 8-neighborhood and selects the next valid foreground pixel that has not yet been visited. If multiple candidates exist, endpoint or junction pixels are preferred over regular foreground pixels. Among such candidates, the closest one is selected according to Manhattan distance. This rule improves local continuity of the traced polyline and reduces short redundant detours that may otherwise arise in the discrete pixel representation.

Algorithm 1: Topology tracing into ordered polylines

```
1:  procedure TRACE_TOPOLOGY(foreground_pixels, endpoints, junctions)
2:      visited ← map initialized to false
3:      result ← empty list of polylines
4:
5:      for all endpoint from endpoints do
6:          if not visited[ endpoint ] then
7:              trace a polyline starting at endpoint
8:              stop when another endpoint or a junction is reached
9:              mark regular traced pixels as visited
10:             add the resulting polyline to result
11:         end if
12:     end for
13:
14:     for all junction from junctions do
15:         for all unvisited foreground neighbors of junction do
16:             trace a polyline starting at junction
17:             stop when another junction is reached
18:             mark regular traced pixels as visited
19:             add the resulting polyline to result
20:         end for
21:     end for
22:
23:     for all unvisited foreground pixels p do
24:         trace a closed contour starting at p
25:         continue until no unvisited foreground neighbor exists
26:         append the starting point again to represent closure
```

```
        add the resulting polyline to result
    end for
    return result
end procedure
```

Tracing from endpoints. In the first stage, each unvisited endpoint is used as a starting point of a new polyline. The tracing then proceeds through unvisited foreground pixels until either another endpoint or a junction is reached. Regular foreground pixels are appended to the polyline and marked as visited. If the traced branch terminates at a junction, the junction is appended to the polyline but is not marked as visited, since it may also serve as a connection point for other branches.

Tracing between junctions. After all endpoint-based branches have been processed, some foreground pixels may still remain unvisited. In a skeleton topology, these pixels are typically expected to form branches connecting one junction to another. Since a single junction may have multiple outgoing branches, each of its unvisited foreground neighbors is considered separately. For each such neighbor, a new polyline is initialized with the starting junction as its first point and traced until another junction is reached. As in the previous case, only regular foreground pixels are marked as visited during the traversal.

Tracing closed contours. After the previous two stages, any remaining unvisited foreground pixels are assumed to belong to closed contours with no endpoints and no junctions. This case occurs most commonly when tracing contours of foreground regions rather than binary mask skeletons. The tracing starts from any remaining unvisited foreground pixel and continues through unvisited foreground neighbors until the contour has been fully traversed. To represent closure explicitly, the starting point is appended once more at the end of the resulting polyline so that the first and last point are identical.

Edge Cases

There are several edge cases that can be handled by the previously described algorithms, but not in an optimal way. In this application, which also relies heavily on user input, we decided not to handle these cases automatically, as doing so may produce unwanted results. Instead, we provide tools that allow the user to resolve such cases manually, for example by stitching two or more polylines together or by closing a polyline.

Examples of such edge cases include the previously mentioned junction plateaus and L-shaped corners. Although the algorithm can process these cases, it may create redundant line segments or unnecessary connections. Another encountered edge case arises when a 2-pixel-wide contour is created from a foreground region that is too thin. This may result in many short line segments and in a polyline that is not closed.

2.4 Polyline Post-processing

Polyline Stitching Algorithm

Polyline stitching is the process of connecting two or more polylines by merging pairs of endpoints, resulting in one or more longer continuous polylines. A distance threshold can be specified that determines when two endpoints are considered close enough to be merged.

The proposed algorithm follows a greedy strategy. In each iteration, it considers all endpoints of currently active polylines and finds the closest valid pair whose distance does not exceed the specified threshold. The corresponding polylines are then merged into a new polyline, while one of the original polylines is marked as inactive. This process is repeated until no further valid merge can be found.

Since the algorithm is greedy in nature, it does not guarantee a optimal stitching configuration. Its intended use-case is to stitch two or more polylines into one continuous polyline.

Algorithm 2: Polyline Stitching algorithm

```
1:  procedure StitchPolylines(polylines, max_distance)
2:      resulting_polylines ← initialize empty list
3:      used_polylines ← initialize list with false values, length of polylines
4:      polyline_ends ← initialize empty list of points
5:
6:      while true do
7:          polyline_ends.reset()
8:
9:          for polyline from polylines do
10:             if not used_polylines[ polyline ] then
11:                 polyline_ends.add(polyline.first)
12:                 polyline_ends.add(polyline.last)
13:             end if
14:         end for
15:
16:         best_pair ← iniitalize
17:         best_distance = ∞
18:         for first_end from polyline_ends do
19:             for second_end from polyline_ends do
20:                 if if both ends are not from the same polyline then
21:                     distance = ||first_end − second_end||
22:
23:                     if distance ≤ max_distance and distance < best_distance then
24:                         best_pair = (first_end, second_end)
25:                         best_distance = distance
26:                     end if
27:                 end if
28:             end for
29:         end for
30:         if best_pair not chosen then
31:             break
32:         end if
33:
34:         construct merged_polyline from the best_pair
35:         reverse one polyline if necessary
36:         remove the duplicate connecting endpoint
37:
38:         polylines[ index of best_pair.first ] = merged_polyline
39:         used_polylines[ best_pair.second] = true
```

```
    end while

    for polyline from polylines do
        if not used_polylines[ polyline] then
            resulting_polylines.add(polyline)
        end if
    end for
    return resulting_polylines
end procedure
```

Depending on which endpoints are connected, one of the polylines may need to be reversed before merging in order to preserve a consistent point ordering. When polylines are merged, one of the connecting endpoints must be removed, as it would result in a duplicate point and other algorithms may not work as intended.

In the implementation, an additional iteration limit can be used as a safety measure to prevent unexpected infinite execution in degenerate cases.

Polyline Simplification Using Douglas–Peucker Algorithm

Polyline created from binary mask skeleton or contour can have large amount of redundant points, which could hinder subsequent spline interpolation. To solve this, we employ Douglas–Peucker algorithm [5] for simplifying polylines.

Distance parameter $\epsilon > 0$ is chosen. The algorithm is initially given first and last point of the polyline and both of them are marked to be kept. Line segment is formed between them. It then examinesall the remaining points and finds with the maximum distance to this line segment. If this distance is greater than ϵ, the point is marked to be kept and the algorithm is recursively applied to the two sub-polylines defined by the first point, the selected point, and the last point

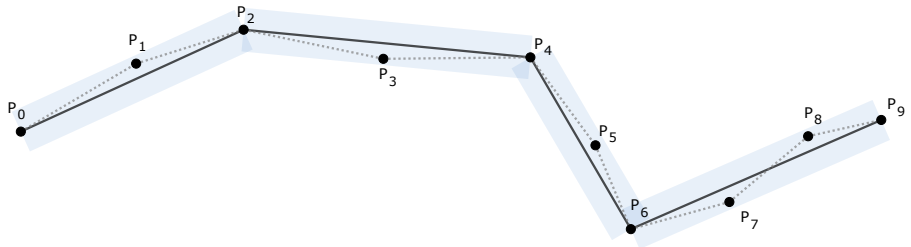


Figure 2.5: Visual example of Douglas–Peucker algorithm. Initial polyline is drawn in gray dotted line, final polyline is black. Blue rectangles are visualization of parameter ϵ.

When the recursion is completed a new polylines is constructed from the points which are marked to be kept. Resulting polyline is of the same orientation and approximates the shape of the initial polyline, depending on the parameter ϵ. This simplification step reduces the number of control points while preserving the overall shape of the extracted topology, which makes the subsequent spline construction more stable and easier to control.

2.5 Spline Construction from Polylines

Hermite and Bézier Curve Representations

In our implementation, we chose to represent splines as a sequence of cubic Bézier curves rather than as Hermite curves, which are used internally by Unreal Engine 5. The main reason for this choice is that Bézier curves are more convenient for the algorithms introduced later in this chapter. At the same time, a cubic Bézier curve can be converted to a Hermite representation in a straightforward manner.

A cubic Bézier curve $\mathbf{B}(t)$ is defined by two end points, $\mathbf{P}_0$ and $\mathbf{P}_3$, and two control points, $\mathbf{P}_1$ and $\mathbf{P}_2$. It is given by

$$\mathbf{B}(t) = (1-t)^3\mathbf{P}_0 + 3(1-t)^2 t\mathbf{P}_1 + 3(1-t)t^2\mathbf{P}_2 + t^3\mathbf{P}_3, \tag{2.11}$$

for $t \in [0, 1]$.

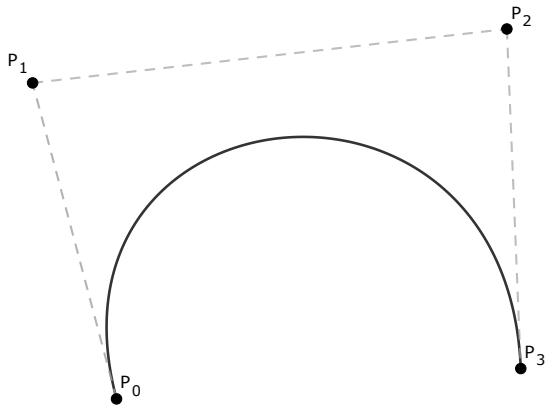


Figure 2.6: Example of cubic Bézier curve constructed from four control points.

A Hermite curve $\mathbf{H}(t)$ is defined by two end points, $\mathbf{H}_0$ and $\mathbf{H}_1$, and two tangent vectors, $\mathbf{M}_0$ and $\mathbf{M}_1$. It is given by

$$\mathbf{H}(t) = (2t^3 - 3t^2 + 1)\mathbf{H}_0 + (t^3 - 2t^2 + t)\mathbf{M}_0 + (-2t^3 + 3t^2)\mathbf{H}_1 + (t^3 - t^2)\mathbf{M}_1, \tag{2.12}$$

for $t \in [0, 1]$.

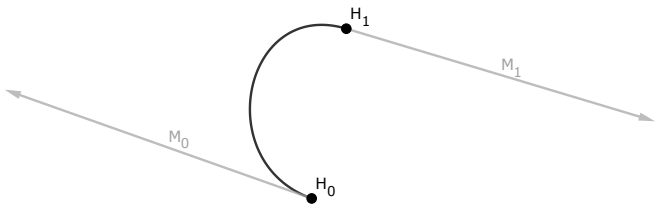


Figure 2.7: Example of Hermite spline constructed from two points and two tangent vectors.

A Hermite curve defined by end points $\mathbf{H}_0$, $\mathbf{H}_1$ and tangent vectors $\mathbf{M}_0$, $\mathbf{M}_1$ can be constructed from a cubic Bézier curve defined by control points $\mathbf{P}_0$, $\mathbf{P}_1$, $\mathbf{P}_2$, and $\mathbf{P}_3$ using the

following relationships:

$$\mathbf{H}_0 = \mathbf{P}_0, \tag{2.13}$$
$$\mathbf{M}_0 = 3\left(\mathbf{P}_1 - \mathbf{P}_0\right), \tag{2.14}$$
$$\mathbf{M}_1 = 3\left(\mathbf{P}_3 - \mathbf{P}_2\right), \tag{2.15}$$
$$\mathbf{H}_1 = \mathbf{P}_3. \tag{2.16}$$

Catmull–Rom Spline Construction

To interpolate spline through points of a polyline, we employed a Catmull–Rom spline. Catmull–Rom splines are a family of cubic interpolating splines constructed such that a tangent at point $\mathbf{P}_i$ is computed using the previous point $\mathbf{P}_{i-1}$ and the next point $\mathbf{P}_{i+1}$.

We used a class of parameterization described in [4] of Catmull–Rom spline. Let the knot sequence satisfy

$$t_{k+1} = t_k + \|\mathbf{P}_{k+1} - \mathbf{P}_k\|^{\alpha}, \tag{2.17}$$

where $\alpha \in [0, 1]$ and $t_0 = 0$. If $\alpha = 0$ then the parameterization is uniform, for $\alpha = 1$ the parameterization becomes chordal.

In our implementation, we use $\alpha = 0.5$, which corresponds to the centripetal parameterization of Catmull–Rom spline. This variant was selected because it is the only parameterization in this family that guarantees no cusps or self-intersections within a single curve segment [6].

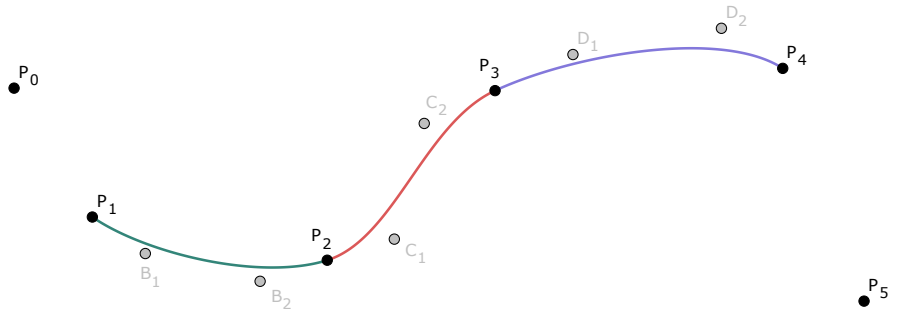


Figure 2.8: Example of spline interpolation by Catmull–Rom spline. Points $\mathbf{B}_1, \mathbf{B}_2, \mathbf{C}_1, \mathbf{C}_2, \mathbf{D}_1, \mathbf{D}_2$ are corresponding control points for Bézier curve.

As follows from the definition, a Catmull–Rom spline segment is constructed from four points, $\mathbf{P}_0$ to $\mathbf{P}_3$, and produces a curve segment between $\mathbf{P}_1$ and $\mathbf{P}_2$. For an open polyline, this means that the spline would not naturally interpolate the first and the last point of the polyline. To resolve this issue, we introduce auxiliary end points defined as

$$\mathbf{P}_{-1} = 2\mathbf{P}_0 - \mathbf{P}_1, \tag{2.18}$$
$$\mathbf{P}_{n+1} = 2\mathbf{P}_n - \mathbf{P}_{n-1}, \tag{2.19}$$

where the polyline consists of points $\mathbf{P}_0, \ldots, \mathbf{P}_n$.

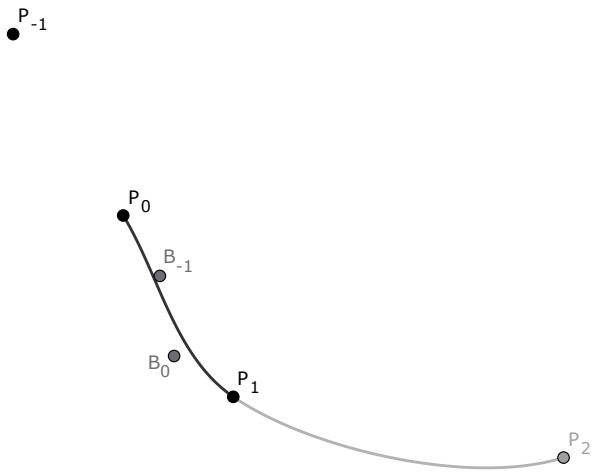


Figure 2.9: Example of Catmull–Rom endpoint extension.

A drawback of this endpoint extension is that it may produce undesirable shapes when the polyline forms a sharp turn near the boundary. As shown in Fig. 2.10, the first spline segment is constructed from the points $\mathbf{P}_{-1}, \mathbf{P}_0, \mathbf{P}_1$, and $\mathbf{P}_2$. Since $\mathbf{P}_{-1}$ is defined to be collinear with $\mathbf{P}_0$ and $\mathbf{P}_1$, the tangent of the segment between $\mathbf{P}_0$ and $\mathbf{P}_1$ at $\mathbf{P}_0$ is aligned with the line passing through these points. As a result, the curve segment is pulled in the tangent direction, which may lead to an unnatural shape near the endpoint. However, since the base polylines are constructed from binary mask skeletons or contours, this situation occurs only rarely in practice.

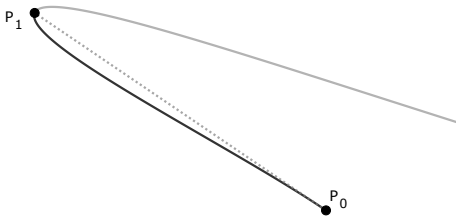


Figure 2.10: Example of Catmull–Rom endpoint extension sharp curve.

The knot sequence in Equation (2.17) determines the parameter spacing between interpolation points. Unlike the Hermite or Bézier formulations, it does not directly provide a polynomial expression of the curve segment. However, this parameterization can be used to construct equivalent Hermite or Bézier representations of the same curve segment.

The Bézier curve representation of a Catmull–Rom spline segment can be computed directly from the four defining points, $\mathbf{P}_0$ to $\mathbf{P}_3$. If the Bézier curve is defined by control

points $\mathbf{B}_0$ to $\mathbf{B}_3$, the relationship is given by [6]

$$\mathbf{B}_0 = \mathbf{P}_1, \tag{2.20}$$

$$\mathbf{B}_1 = \frac{d_1^{2\alpha}\mathbf{P}_2 - d_2^{2\alpha}\mathbf{P}_0 + \left(2d_1^{2\alpha} + 3d_1^{\alpha}d_2^{\alpha} + d_2^{2\alpha}\right)\mathbf{P}_1}{3d_1^{\alpha}\left(d_1^{\alpha} + d_2^{\alpha}\right)}, \tag{2.21}$$

$$\mathbf{B}_2 = \frac{d_3^{2\alpha}\mathbf{P}_1 - d_2^{2\alpha}\mathbf{P}_3 + \left(2d_3^{2\alpha} + 3d_3^{\alpha}d_2^{\alpha} + d_2^{2\alpha}\right)\mathbf{P}_2}{3d_3^{\alpha}\left(d_3^{\alpha} + d_2^{\alpha}\right)}, \tag{2.22}$$

$$\mathbf{B}_3 = \mathbf{P}_2, \tag{2.23}$$

where d_1, d_2, and d_3 are defined as

$$d_1 = \|\mathbf{P}_1 - \mathbf{P}_0\|, \tag{2.24}$$

$$d_2 = \|\mathbf{P}_2 - \mathbf{P}_1\|, \tag{2.25}$$

$$d_3 = \|\mathbf{P}_3 - \mathbf{P}_2\|, \tag{2.26}$$

and $\|\cdot\|$ denotes the Euclidean norm. The knot intervals corresponding to the distances d_1, d_2, and d_3 satisfy

$$t_1 - t_0 = d_1^{\alpha}, \tag{2.27}$$

$$t_2 - t_1 = d_2^{\alpha}, \tag{2.28}$$

$$t_3 - t_2 = d_3^{\alpha}. \tag{2.29}$$

Curve Subdivision Using de Casteljau's Algorithm

Bézier curve can be constructed recursively by using de Casteljau's algorithm. This method is numerically more stable, than using polynomial definition (2.11). Anotherimportant use case of this algorithm is subdividing Bézier curve into two segments and subsequently getting correct their control points.

Let cubic Bézier curve $\mathbf{B}(t)$ be defined by four control points $\mathbf{P}_0$ to $\mathbf{P}_3$ and $t \in [0, 1]$. De Casteljau's algorithm for cubic Bézier curve $\mathbf{B}$ works by performing three steps. For parameter $u \in [0, 1]$ it creates points

$$\mathbf{Q}_i = (\mathbf{P}_{i+1} - \mathbf{P}_i)u + \mathbf{P}_i, \tag{2.30}$$

for $i = 0, 1, 2$. Point $\mathbf{Q}_i$ is a point on a line connecting control points $\mathbf{P}_i$ an $\mathbf{P}_{i+1}$. Then it creates another points

$$\mathbf{R}_j = (\mathbf{Q}_{j+1} - \mathbf{Q}_j)u + \mathbf{Q}_j, \tag{2.31}$$

for $j = 0, 1$. Similarly as with $\mathbf{Q}_i$, point $\mathbf{R}_j$ is a point on a line connecting points $\mathbf{Q}_j$ and $\mathbf{Q}_{j+1}$. Final point is constructed on a line between $\mathbf{R}_0$ and $\mathbf{R}_1$ as

$$\mathbf{S} = (\mathbf{R}_1 - \mathbf{R}_0)u + \mathbf{R}_0. \tag{2.32}$$

Final point $\mathbf{S}$ lies on a cubic Bézier $\mathbf{B}(t)$ and is equal to a point $\mathbf{B}(u)$.

We can then create two cubic Bézier curves $\mathbf{B}_A(t_A)$ and $\mathbf{B}_B(t_B)$, and $t_A, t_B \in [0, 1]$. Bézier curve $\mathbf{B}_A$, visualized in 2.12a, is defined by control points $\mathbf{P}_0$, $\mathbf{Q}_0$, $\mathbf{R}_0$ and $\mathbf{S}$. Bézier curve $\mathbf{B}_B$, visualized in 2.12b, is defined by control points $\mathbf{S}$, $\mathbf{R}_1$, $\mathbf{Q}_2$ and $\mathbf{P}_3$. These segments when combined results in the initial Bézier curve $\mathbf{B}(t)$.

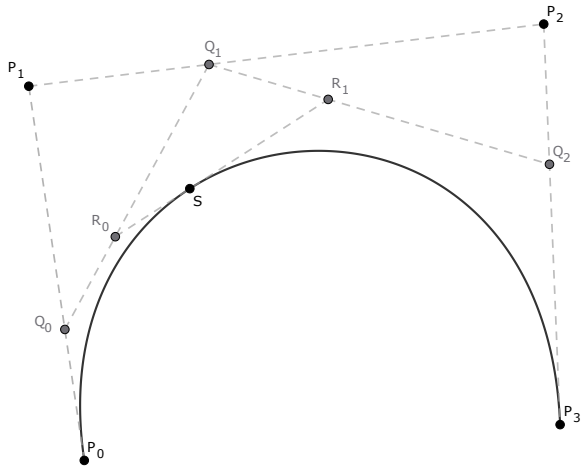
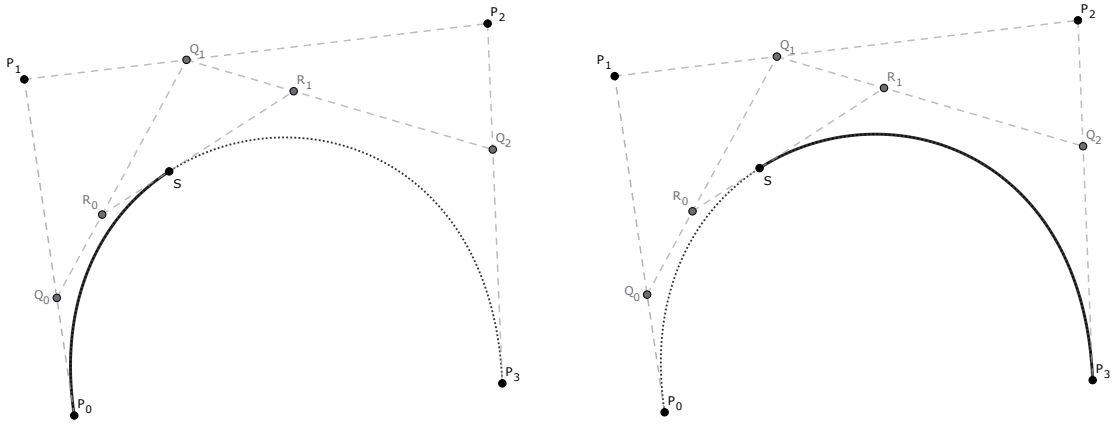


Figure 2.11: Visual example of de Casteljau algorithm for $u = 0.35$.



(a) Left spline subdivision.

(b) Right spline subdivision.

Figure 2.12: Example of subdividing Bézier curve using de Casteljau algorithm for $u = 0.35$, the left subdivion (a) and the right subdivision (b).

3 Conclusion

Summarize the achieved results. Can be similar as an abstract or an introduction, however, it should be written in past tense.

4 References

[1] H. Jain and A. P. Kumar, *A Sequential Thinning Algorithm For Multi-Dimensional Binary Patterns*, en, Oct. 2017. Accessed: Mar. 29, 2026. [Online]. Available: https://arxiv.org/abs/1710.03025v2

[2] P. F. Felzenszwalb and D. P. Huttenlocher, "Distance Transforms of Sampled Functions," *Theory of Computing*, vol. 8, pp. 415–428, Sep. 2012, Number: 19. DOI: 10.4086/toc.2012.v008a019 Accessed: Mar. 29, 2026. [Online]. Available: https://theoryofcomputing.org/articles/v008a019/

[3] Z. Guo and R. W. Hall, "Fast fully parallel thinning algorithms," *CVGIP: Image Understanding*, vol. 55, no. 3, pp. 317–328, May 1992, ISSN: 1049-9660. DOI: 10.1016/1049-9660(92)90029-3 Accessed: Mar. 29, 2026. [Online]. Available: https://www.sciencedirect.com/science/article/pii/1049966092900293

[4] E. T. Y. Lee, "Choosing nodes in parametric curve interpolation," *Computer-Aided Design*, vol. 21, no. 6, pp. 363–370, Jul. 1989, ISSN: 0010-4485. DOI: 10.1016/0010-4485(89)90003-1 Accessed: Apr. 9, 2026. [Online]. Available: https://www.sciencedirect.com/science/article/pii/0010448589900031

[5] D. H. Douglas and T. K. Peucker, *ALGORITHMS FOR THE REDUCTION OF THE NUMBER OF POINTS REQUIRED TO REPRESENT A DIGITIZED LINE OR ITS CARICATURE | Cartographica*. Accessed: Mar. 29, 2026. [Online]. Available: https://utppublishing.com/doi/abs/10.3138/fm57-6770-u75u-7727

[6] C. Yuksel, S. Schaefer, and J. Keyser, *On the parameterization of Catmull-Rom curves | 2009 SIAM/ACM Joint Conference on Geometric and Physical Modeling*. Accessed: Mar. 29, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/1629255.1629262

[7] T. Y. Zhang and C. Y. Suen, *A fast parallel algorithm for thinning digital patterns | Communications of the ACM*. Accessed: Mar. 29, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/357994.358023

A Appendix A